

Introduction à la programmation orientée objet

En JavaScript, la programmation orientée objet permet de structurer le code en utilisant des objets qui encapsulent (englobent) des données et des comportements. Une **classe** est un modèle pour créer des objets avec des propriétés et des méthodes spécifiques. On appelle **instance** un objet créé à partir d'une classe.

Voyez le fichier de classe comme un plan de maison. Il décrit comment la maison doit être construite, mais ce n'est pas la maison elle-même. Chaque fois que vous utilisez ce plan pour construire une maison, vous créez une **instance** de cette maison avec le mot-clé **new**.

Exemple

```
class Voiture {
  constructor(marque, modele, annee) {
    this.marque = marque;
    this.modele = modele;
    this.annee = annee;
  }

  afficherDetails() {
    console.log(`Voiture: ${this.marque} ${this.modele}, Année:
    ${this.annee}`);
  }
}

// Création d'une instance de la classe Voiture
const maVoiture = new Voiture("Toyota", "Corolla", 2020);
maVoiture.afficherDetails(); // Affiche: Voiture: Toyota Corolla, Année: 2020

class Bouton {
  constructor(conteneurHTML, label, couleur) {
    this.conteneurHTML = conteneurHTML;
    this.label = label;
    this.couleur = couleur;
    this.generer();
  }

  generer() {
    const bouton = document.createElement("button");

    bouton.textContent = this.label;
    bouton.style.backgroundColor = this.couleur;
    bouton.addEventListener("click", this.cliquer.bind(this));

    this.conteneurHTML.appendChild(bouton);
  }

  cliquer() {
    console.log(`Le bouton ${this.label} a été cliqué!`);
  }
}
```

```
}  
}  
  
const premierBouton = new Bouton(document.body, "Cliquez-moi", "blue");
```

Pourquoi utiliser la programmation orientée objet?

La programmation orientée objet offre plusieurs avantages :

1. **Réutilisabilité** : Le code est situé à un seul endroit et peut être appelé partout dans l'application. Dans l'exemple plus haut, la classe `Bouton` peut être réutilisée pour créer autant de boutons que nécessaire avec la même interactivité.
2. **Modularité** : Le code est organisé en modules (classes) qui peuvent être développés et testés indépendamment. Le code est généralement divisé en plusieurs fichiers, chaque fichier contenant une classe ou un ensemble de classes liées. (Cours 1)
3. **Encapsulation** : Les détails internes d'une classe sont cachés, ce qui protège l'intégrité des données. Nous utiliserons des propriétés et méthodes privées (Cours 4)
4. **Héritage** : Les classes peuvent hériter des propriétés et des méthodes d'autres classes, facilitant ainsi l'extension du code. Cela permet de faire des sous-classes spécialisées à partir de classes plus générales. Par exemple, nous avons une classe de base `Bouton`, et nous pourrions créer une sous-classe `BoutonImage` qui hérite de `Bouton` et ajoute des fonctionnalités spécifiques aux boutons avec des images. (Cours 5)
5. **Polymorphisme** : Les classes peuvent être traitées comme des instances de leur classe parente, ce qui permet d'utiliser des méthodes de manière interchangeable. Par exemple, la méthode `cliquer` pourrait être redéfinie dans une sous-classe pour offrir un comportement différent de la classe de base tout en étant appelée de la même manière.

Créer un fichier de classe

Pour créer un fichier de classe, ajoutez le mot-clé `class` suivi du nom de la classe et des accolades contenant le code de la classe. **Pas de parenthèses comme avec les fonctions.**

Ensuite, ajoutez une fonction spéciale appelée `constructor` qui sera exécutée lors de la création d'une instance de la classe. Cette fonction peut prendre des paramètres pour initialiser les propriétés de l'objet lorsqu'on le crée avec `new`.

Finalement, exporter la classe pour pouvoir l'importer dans d'autres fichiers.

```
// Boutons.js  
  
class Bouton {  
  constructor(conteneurHTML, label, couleur) {  
    this.conteneurHTML = conteneurHTML;  
    this.label = label;  
    this.couleur = couleur;  
    this.element = null;  
    this.generer();  
  }  
}
```

```

generer() {
    const bouton = document.createElement("button");

    bouton.textContent = this.label;
    bouton.style.backgroundColor = this.couleur;
    bouton.addEventListener("click", this.cliquer.bind(this));
    this.element = bouton;
    this.conteneurHTML.appendChild(bouton);
}

cliquer() {
    console.log(`Le bouton ${this.label} a été cliqué!`);
}

afficherDetails() {
    console.log(`Bouton: ${this.label}, Couleur: ${this.couleur}`);
}

modifierLabel(nouveauLabel) {
    this.label = nouveauLabel;
    if (this.element) {
        this.element.textContent = nouveauLabel;
    }
}
}

export default Bouton;

```

Les propriétés et les méthodes

Les **propriétés** sont des variables attachées à une instance de la classe qui stockent des données spécifiques à cette instance. Elles sont définies dans le constructeur en utilisant `this`.

Les **méthodes** sont des fonctions définies à l'intérieur de la classe qui opèrent sur les données de l'instance. Elles peuvent accéder et modifier les propriétés de l'instance en utilisant `this`. Voyez cela comme des actions possibles que l'objet peut effectuer. Toutes les instances partagent les mêmes méthodes, mais chaque instance a ses propres propriétés.

Dans cet exemple, la classe `LightBox` a trois propriétés : `conteneurHTML`, `listeImages` et `index`. Elle a aussi une méthode appelée `generer` qui sera utilisée pour créer l'interface utilisateur de la lightbox qui sera appelée dans le constructeur.

```

class LightBox {
    constructor(conteneurHTML, listeImages) {
        this.conteneurHTML = conteneurHTML;
        this.listeImages = listeImages;
        this.index = 0;
        this.element = null;
        this.generer();
    }

    generer() {

```

```

    // Code pour injecter le code HTML sur la page
    //...
    this.afficherImage(this.index);
  }
  afficherImage(index) {
    // Code pour afficher une image spécifique
  }
}

```

Importer une classe et l'instancier

Une instance, comme indiqué précédemment, est créée en utilisant le mot-clé **new** suivi du nom de la classe et des parenthèses contenant les arguments nécessaires pour le constructeur. Nous pouvons créer autant d'instances que nécessaire et il est possible de les stocker dans des variables afin de les manipuler ultérieurement.

Chaque instance est indépendante des autres, ce qui signifie que les modifications apportées à une instance n'affecteront pas les autres instances.

```

// main.js
import LightBox from "./LightBox.js";
let maLightBox1 = new LightBox(document.body, ["img1.jpg", "img2.jpg",
"img3.jpg"]);
maLightBox1.afficherImage(2);

let maLightBox2 = new LightBox(document.body, ["img4.jpg", "img5.jpg",
"img6.jpg"]);
maLightBox2.afficherImage(1);

```

this

Pour accéder aux propriétés et méthodes d'une instance à l'intérieur de la classe, on utilise le mot-clé **this**. **This** fait référence à l'instance actuelle de la classe. Cela permet d'accéder aux données spécifiques à cette instance. Lors du constructeur, on enregistre les informations passées en paramètres dans des propriétés de l'instance en utilisant **this**. Elles sont ensuite accessibles dans toutes les méthodes de la classe via **this**.

```

class Bouton {
  constructor(conteneurHTML, label, couleur) {
    //L'étape ci-dessous est essentielle afin d'enregistrer les différentes
    propriétés de l'instance de l'objet
    this.conteneurHTML = conteneurHTML;
    this.label = label;
    this.couleur = couleur;
    this.element = null;
    this.generer();
  }

  afficherDetails() {
    console.log(`Bouton: ${this.label}, Couleur: ${this.couleur}`);
  }
}

```

```
}  
}
```

Fichiers de classe, this, bind, addEventListener

Dans un fichier de classe, une erreur fréquente est de perdre le contexte du `this` lors de l'ajout d'écouteurs d'événements ou lors de fonctions de haut niveau car `this` est toujours associé au contexte englobant, c'est-à-dire qu'utiliser `this` dans la fonction passée en paramètre du `addEventListener`, d'un `map`, d'un `forEach`, etc., n'est pas le même contexte que le `this` de la classe.

Pour éviter cela, on utilise la méthode `bind` pour lier le contexte de `this` à l'instance de la classe.

```
class Bouton {  
  constructor(conteneurHTML, label, couleur) {  
    this.conteneurHTML = conteneurHTML;  
    this.label = label;  
    this.couleur = couleur;  
    this.element = null;  
    this.generer();  
  }  
  
  generer() {  
    const gabarit = `${this.label}</button>`;   
    this.conteneurHTML.insertAdjacentHTML("beforeend", gabarit);  
  
    //On enregistre l'élément ajouté pour y ajouter un écouteur d'événement  
    plus tard  
    this.element = this.conteneurHTML.lastElementChild;  
    //Si on n'utilise pas bind, this ne fera pas référence à l'instance de la  
    classe  
    this.element.addEventListener("click", this.cliquer.bind(this));  
  }  
  
  cliquer() {  
    //Sans bind, impossible d'accéder aux autres propriétés de l'instance  
    console.log(`Le bouton ${this.label} a été cliqué!`);  
  }  
}
```

Une autre façon de faire est d'utiliser les fonctions fléchées car leur contexte de `this` est toujours celui de l'instance de la classe.

```
class Bouton {  
  constructor(conteneurHTML, label, couleur = "red") {  
    this.conteneurHTML = conteneurHTML;  
    this.label = label;  
    this.couleur = couleur;  
    this.element = null;  
    this.generer();  
  }  
}
```

```

    generer = () => {
        const gabarit = `<button data-couleur=${this.couleur}>${this.label}</button>`;
        this.conteneurHTML.insertAdjacentHTML("beforeend", gabarit);

        //On enregistre l'élément ajouté pour y ajouter un écouteur d'événement
        plus tard
        this.element = this.conteneurHTML.lastElementChild;
        //Si on n'utilise pas bind, this ne fera pas référence à l'instance de la
        classe
        this.element.addEventListener("click", this.cliquer);
    };

    cliquer = () => {
        //Sans bind, impossible d'accéder aux autres propriétés de l'instance
        console.log(`Le bouton ${this.label} a été cliqué!`);
    };
}

new Bouton(document.body, "patate");

```

Stratégies d'injection de composants HTML

Lors de la création de composants réutilisables avec des classes, il est important de choisir une stratégie d'injection du code HTML dans la page. Dans le constructeur, on peut passer en paramètre un conteneur HTML (un élément du DOM) dans lequel le composant sera injecté. Cela permet de contrôler où le composant apparaîtra dans la page et rendre votre classe plus réutilisable

Ayez une propriété `elementHTML` qui peut être null lors de l'instanciation et qui sera utilisée pour stocker l'élément HTML du composant une fois qu'il aura été généré. Vous pourrez ainsi y accéder dans les autres méthodes de la classe au besoin.

Ayez une méthode `generer` qui sera appelée dans le constructeur pour créer et injecter le code HTML dans le conteneur spécifié. Cette méthode peut utiliser `insertAdjacentHTML`, `innerHTML`, ou `createElement` selon vos besoins mais privilégiez `insertAdjacentHTML` pour ne pas écraser le contenu existant du conteneur et favoriser la lisibilité. Si le contenu provient de l'utilisateur, assurez-vous de toujours échapper les variables pour réduire les risques d'injection XSS (Voir les notes semaine 1);